

ACTIVITY 1

```
graph = {
    '5' : ['3','7'],
    '3' : ['2', '4'],
    '7' : ['8'],
    '2' : [],
    '4' : ['8'],
    '8' : []
}
visited = [] # List for visited nodes.
queue = []
#Initialize a queue
def bfs(visited, graph, node): #function for BFS
    visited.append(node)
    queue.append(node)
    while queue:
        m = queue.pop(0)
        print (m, end = " ")
    # Creating loop to visit each node
    for neighbour in graph[m]:
        if neighbour not in visited:
            visited.append(neighbour)
            queue.append(neighbour)
# Driver Code
print("Following is the Breadth-First Search")
bfs(visited, graph, '5')
# function calling
```

Following is the Breadth-First Search

✓ ACTIVITY 2

```
def printParents(node, adj, parent):

    # current node is Root, thus, has no parent
    if (parent == 0):
        print(node, "->Root")
    else:
        print(node, "->", parent)

    # Using DFS
    for cur in adj[node]:
        if (cur != parent):
            printParents(cur, adj, node)
```

```
# Function to print the children of each node
def printChildren(Root, adj):

    # Queue for the BFS
    q = []

    # pushing the root
    q.append(Root)

    # visit array to keep track of nodes that have been
    # visited
    vis = [0]*len(adj)

    # BFS
    while (len(q) > 0):
        node = q[0]
        q.pop(0)
        vis[node] = 1
        print(node, "-> ", end=" ")

        for cur in adj[node]:
            if (vis[cur] == 0):
                print(cur, " ", end=" ")
                q.append(cur)
        print("\n")

# Function to print the leaf nodes
def printLeafNodes(Root, adj):

    # Leaf nodes have only one edge and are not the root
    for i in range(0, len(adj)):
        if (len(adj[i]) == 1 and i != Root):
            print(i, end=" ")
    print("\n")

# Function to print the degrees of each node
def printDegrees(Root, adj):

    for i in range(1, len(adj)):
        print(i, ": ", end=" ")

        # Root has no parent, thus, its degree is equal to
        # the edges it is connected to
        if (i == Root):
            print(len(adj[i]))
        else:
            print(len(adj[i])-1)

# Driver code

# Number of nodes
```

```

N = 7
Root = 1

# Adjacency list to store the tree
adj = []
for i in range(0, N+1):
    adj.append([])

# Creating the tree
adj[1].append(2)
adj[2].append(1)

adj[1].append(3)
adj[3].append(1)

adj[1].append(4)
adj[4].append(1)

adj[2].append(5)
adj[5].append(2)
adj[2].append(6)
adj[6].append(2)
adj[4].append(7)
adj[7].append(4)
# Printing the parents of each node
print("The parents of each node are:")
printParents(Root, adj, 0)
# Printing the children of each node
print("The children of each node are:")
printChildren(Root, adj)
# Printing the leaf nodes in the tree
print("The leaf nodes of the tree are:")
printLeafNodes(Root, adj)
# Printing the degrees of each node
print("The degrees of each node are:")
printDegrees(Root, adj)

```

```

The parents of each node are:
1 ->Root
2 -> 1
5 -> 2
6 -> 2
3 -> 1
4 -> 1
7 -> 4
The children of each node are:
1 -> 2 3 4

2 -> 5 6

3 ->

4 -> 7

```

5 ->

6 ->

7 ->

The leaf nodes of the tree are:

3 5 6 7

The degrees of each node are:

1 : 3

2 : 2

3 : 0

4 : 1

5 : 0

6 : 0

7 : 0

✓ LAB TASK 1

```
from collections import deque

# Defining the graph based on the provided image
# Structure: { Node: [(Neighbor, Cost), ...] }
graph = {
    'A': [('B', 6), ('C', 9), ('E', 1)],
    'B': [('A', 6), ('D', 3), ('E', 4)],
    'C': [('A', 9), ('F', 2), ('G', 3)],
    'D': [('B', 3), ('E', 5), ('F', 7)],
    'E': [('A', 1), ('B', 4), ('D', 5), ('F', 6)],
    'F': [('C', 2), ('E', 6), ('D', 7)],
    'G': [('C', 3)]
}

def bfs_with_cost(graph, start, goal):
    # Queue stores (current_node, path_taken)
    queue = deque([(start, [start])])
    visited = set([start])

    while queue:
        (current_node, path) = queue.popleft()

        # Check if we reached the goal
        if current_node == goal:
            # Calculate total cost of the path found
            total_cost = 0
            for i in range(len(path) - 1):
                node_a = path[i]
                node_b = path[i+1]
```

```

        # Find the weight between node_a and node_b
        for neighbor, weight in graph[node_a]:
            if neighbor == node_b:
                total_cost += weight
        return path, total_cost

    # Explore neighbors
    for neighbor, weight in graph[current_node]:
        if neighbor not in visited:
            visited.add(neighbor)
            queue.append((neighbor, path + [neighbor]))

    return None, 0

# Execution
start_node = 'A'
goal_node = 'G'
path, cost = bfs_with_cost(graph, start_node, goal_node)

print(f"Path found by BFS: {' -> '.join(path)}")
print(f"Total cost of this path: {cost}")

```

Path found by BFS: A -> C -> G
Total cost of this path: 12

LAB TASK 2:

```

from collections import deque

# i. Corresponding graph dictionary (Adjacency List with weights)
romania_map = {
    'Arad': [('Zerind', 75), ('Sibu', 140), ('Timisoara', 118)],
    'Zerind': [('Arad', 75), ('Oradea', 71)],
    'Oradea': [('Zerind', 71), ('Sibu', 151)],
    'Sibu': [('Arad', 140), ('Oradea', 151), ('Fagaras', 99), ('Rimnicu Vilcea', 80)],
    'Timisoara': [('Arad', 118), ('Lugoj', 111)],
    'Lugoj': [('Timisoara', 111), ('Mehadia', 70)],
    'Mehadia': [('Lugoj', 70), ('Drobeta', 75)],
    'Drobeta': [('Mehadia', 75), ('Craiova', 120)],
    'Craiova': [('Drobeta', 120), ('Rimnicu Vilcea', 146), ('Pitesti', 138)],
    'Rimnicu Vilcea': [('Sibu', 80), ('Craiova', 146), ('Pitesti', 97)],
    'Fagaras': [('Sibu', 99), ('Bucharest', 211)],
    'Pitesti': [('Rimnicu Vilcea', 97), ('Craiova', 138), ('Bucharest', 101)],
    'Bucharest': [('Fagaras', 211), ('Pitesti', 101), ('Giurgiu', 90), ('Urziceni', 85)],
    'Giurgiu': [('Bucharest', 90)],
    'Urziceni': [('Bucharest', 85), ('Vaslui', 142), ('Hirsova', 98)],
    'Vaslui': [('Urziceni', 142), ('Iasi', 92)],
    'Iasi': [('Vaslui', 92), ('Neamt', 87)],
    'Neamt': [('Iasi', 87)],
    'Hirsova': [('Urziceni', 98), ('Eforie', 86)],
    'Eforie': [('Hirsova', 86)]
}

```

```

    EROPIE : [(HIRSOVA , 80)]
}

# ii. Breadth-First Search Algorithm
def bfs_travelling_salesman(graph, start, goal):
    # Queue stores (current_node, path_taken)
    queue = deque([(start, [start])])
    visited = {start}

    while queue:
        current_node, path = queue.popleft()

        # Goal check
        if current_node == goal:
            # Calculate total distance of the BFS path
            total_distance = 0
            for i in range(len(path) - 1):
                u, v = path[i], path[i+1]
                for neighbor, weight in graph[u]:
                    if neighbor == v:
                        total_distance += weight
            return path, total_distance

        # Expansion
        for neighbor, weight in graph.get(current_node, []):
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append((neighbor, path + [neighbor]))

    return None, 0

# Results
start_city = 'Arad'
goal_city = 'Bucharest'
final_path, total_km = bfs_travelling_salesman(romania_map, start_city, goal_city)

print(f"--- BFS Results ---")
print(f"Path: {' -> '.join(final_path)}")
print(f"Total Distance: {total_km} km")

--- BFS Results ---
Path: Arad -> Sibiu -> Fagaras -> Bucharest
Total Distance: 450 km

```

